

Implementação e Avaliação Experimental de um Protocolo Reliable UDP Baseado em Go-Back-N em Comparação com TCP

Nara Raquel Dias Andrade

¹ Programa de Pós-Graduação em Ciência da Computação (PPGCC)
Universidade Federal do Piauí (UFPI)
Teresina - PI - Brasil

nara.andrade@ufpi.edu.br

Abstract. *The UDP protocol provides fast and lightweight communication but lacks native mechanisms for reliability, packet ordering, and retransmission. This work presents the implementation of a Reliable UDP (R-UDP) protocol based on the Go-Back-N sliding window mechanism, developed in Python and executed in a Docker environment. The proposed protocol was experimentally compared to TCP under different network conditions involving delay and packet loss simulated using NetEm. Metrics such as throughput, transfer time, retransmissions, network overhead, and packet count were evaluated. The results show that TCP consistently outperforms the proposed R-UDP implementation, while the R-UDP protocol experiences significant performance degradation under adverse network conditions due to the increased number of retransmissions required by the Go-Back-N mechanism.*

Resumo. *O protocolo UDP oferece comunicação rápida e de baixa sobrecarga, porém não fornece mecanismos nativos de confiabilidade, ordenação ou retransmissão de pacotes. Este trabalho apresenta a implementação de um protocolo Reliable UDP (R-UDP) baseado no mecanismo de janela deslizante Go-Back-N, desenvolvido em Python e executado em ambiente Docker. O protocolo foi comparado experimentalmente ao TCP sob diferentes condições de atraso e perda de pacotes simuladas por meio da ferramenta NetEm. Foram avaliadas métricas como throughput, tempo de transferência, retransmissões, overhead de rede e quantidade de pacotes observados. Os resultados demonstram que o TCP apresenta desempenho superior em todos os cenários analisados, enquanto o R-UDP sofre degradação significativa à medida que as condições da rede se tornam mais adversas, principalmente devido ao aumento das retransmissões exigidas pelo mecanismo Go-Back-N.*

1. Introdução

A comunicação confiável é um dos principais requisitos de diversas aplicações distribuídas modernas. Protocolos de transporte devem garantir que os dados sejam entregues corretamente ao destinatário, mesmo diante de falhas, perdas de pacotes ou atrasos na rede.

O protocolo TCP (Transmission Control Protocol) é amplamente utilizado por fornecer mecanismos nativos de confiabilidade, incluindo confirmação de recebimento, retransmissão automática, controle de fluxo e controle de congestionamento. Em contrapartida, o protocolo UDP (User Datagram Protocol) oferece uma comunicação mais simples e com menor sobrecarga, porém sem garantias de entrega, ordenação ou integridade dos dados transmitidos. Em determinados cenários, pode ser interessante implementar mecanismos de confiabilidade diretamente na camada de aplicação, permitindo maior controle sobre o comportamento do protocolo. Nesse contexto, surge o conceito de Reliable UDP (R-UDP), no qual funcionalidades normalmente presentes no TCP são implementadas sobre o UDP.

Este trabalho apresenta a implementação de um protocolo R-UDP baseado no mecanismo de janela deslizante Go-Back-N. O protocolo foi desenvolvido em Python e comparado experimentalmente ao TCP sob diferentes condições de rede simuladas. Foram avaliadas métricas relacionadas ao desempenho da aplicação e ao comportamento do tráfego observado na rede. O principal objetivo deste trabalho é analisar o impacto da implementação de mecanismos de confiabilidade na camada de aplicação e comparar seu desempenho com as funcionalidades já oferecidas pelo TCP.

2. Desenvolvimento

2.1. Arquitetura da Solução

O sistema desenvolvido segue uma arquitetura cliente-servidor implementada em Python e executada em ambiente Docker. Foram construídas duas soluções distintas para transferência de arquivos: uma baseada no protocolo TCP e outra baseada em um protocolo Reliable UDP (R-UDP) desenvolvido sobre o protocolo UDP. A aplicação é composta por dois containers principais: um cliente responsável pelo envio dos arquivos e um servidor responsável pelo recebimento e armazenamento dos dados transmitidos. Durante os experimentos, os containers foram conectados por uma rede virtual Docker, permitindo a simulação de diferentes condições de atraso e perda de pacotes.

Além dos containers de aplicação, foram utilizadas ferramentas auxiliares para captura e análise do tráfego de rede. O tcpdump foi empregado para registrar os pacotes trafegados durante cada execução, enquanto o tshark foi utilizado posteriormente para extrair métricas quantitativas a partir dos arquivos PCAP gerados.

2.2. Implementação do Protocolo TCP

A implementação TCP utiliza os mecanismos de confiabilidade já fornecidos pelo sistema operacional. O cliente estabelece uma conexão com o servidor e transmite o conteúdo do arquivo em blocos de tamanho fixo até que todos os dados sejam enviados. Antes da transmissão dos dados, são enviados metadados contendo o nome do arquivo, tamanho total e um identificador de autenticação definido pelo projeto. Ao final da transferência, o servidor verifica a quantidade de bytes recebidos e armazena o arquivo reconstruído no diretório de saída. As métricas de tempo de execução e throughput foram coletadas diretamente pela aplicação cliente durante cada experimento.

2.3. Implementação do Protocolo R-UDP

O protocolo R-UDP foi implementado sobre sockets UDP utilizando mecanismos adicionais para garantir a confiabilidade da transmissão. Cada pacote transmitido possui um

cabeçalho contendo tipo do pacote, número de sequência, checksum e carga útil. Foram definidos três tipos principais de pacotes: metadados, dados e encerramento da transmissão.

A confiabilidade foi implementada por meio do algoritmo Go-Back-N, utilizando janela deslizante e confirmações cumulativas. O transmissor mantém uma janela de pacotes pendentes e aguarda o recebimento de ACKs enviados pelo receptor. Caso ocorra timeout, todos os pacotes a partir da base da janela são retransmitidos. Para garantir a integridade dos dados, cada pacote possui um checksum calculado antes do envio e validado pelo receptor. Pacotes corrompidos são descartados e retransmitidos posteriormente pelo mecanismo de recuperação de perdas.

Além dos dados do arquivo, o protocolo também transmite metadados contendo nome do arquivo, tamanho esperado e o campo de autenticação exigido pelas especificações do projeto.

2.4. Ambiente Experimental

Todos os experimentos foram executados em ambiente Docker utilizando containers independentes para cliente e servidor. A captura do tráfego de rede foi realizada com a ferramenta tcpdump, gerando arquivos PCAP para cada execução. Posteriormente, esses arquivos foram analisados utilizando tshark para extração de métricas relacionadas ao tráfego observado na rede.

As condições da rede foram controladas por meio da ferramenta NetEm, permitindo a configuração de atrasos artificiais e perdas de pacotes. Foram executadas cinco repetições para cada cenário experimental, totalizando trinta execuções considerando os protocolos TCP e R-UDP.

3. Metodologia Experimental

3.1. Cenários de Teste

Para avaliar o comportamento dos protocolos TCP e R-UDP sob diferentes condições de rede, foram definidos três cenários experimentais utilizando a ferramenta NetEm. Os cenários simulam diferentes níveis de atraso e perda de pacotes, permitindo analisar o impacto dessas condições sobre o desempenho da comunicação.

A Tabela 1 apresenta os cenários utilizados durante os experimentos.

Tabela 1. Cenários experimentais utilizados

Cenário	Atraso	Perda de Pacotes
A	10 ms	0%
B	50 ms	10%
C	100 ms	20%

O Cenário A representa uma condição próxima do ideal, sem perdas e com baixa latência. O Cenário B introduz degradação moderada da rede, enquanto o Cenário C representa uma situação mais severa, com elevado atraso e perda significativa de pacotes.

Para cada cenário foram realizadas cinco execuções independentes para cada protocolo, totalizando trinta experimentos.

3.2. Arquivo de Teste

Os experimentos foram realizados utilizando um arquivo binário de 512 KB gerado artificialmente com dados aleatórios. O mesmo arquivo foi utilizado em todas as execuções, garantindo condições equivalentes para comparação entre os protocolos.

Durante cada transmissão foram registrados os tempos de execução, métricas da aplicação e métricas observadas diretamente na rede.

3.3. Captura e Análise do Tráfego

O tráfego gerado durante cada experimento foi capturado utilizando a ferramenta *tcpdump*. Cada execução produziu um arquivo PCAP contendo todos os pacotes observados durante a transferência.

Posteriormente, os arquivos PCAP foram analisados utilizando a ferramenta *tshark*, permitindo extrair métricas relacionadas ao comportamento da rede, como quantidade de pacotes transmitidos, volume total de dados trafegados e throughput observado.

As métricas obtidas foram armazenadas em arquivos CSV e posteriormente processadas utilizando Python para geração dos gráficos e tabelas apresentados neste relatório.

3.4. Métricas Avaliadas

Foram coletadas métricas tanto no nível da aplicação quanto no nível da rede.

A Tabela 2 resume as métricas analisadas.

Tabela 2. Métricas avaliadas

Métrica	Descrição
Throughput	Taxa de transferência de dados úteis da aplicação (bytes por segundo).
Tempo de Transferência	Tempo total necessário para concluir o envio do arquivo.
Retransmissões	Quantidade de retransmissões realizadas pelo protocolo R-UDP.
Overhead	Quantidade adicional de bytes trafegados na rede além do tamanho original do arquivo.
Pacotes Observados	Número total de pacotes capturados durante a transferência.
Throughput da Rede	Taxa de transferência calculada a partir dos pacotes observados nos arquivos PCAP.

As métricas foram analisadas por meio de médias e desvios padrão calculados a partir das cinco execuções realizadas em cada cenário.

4. Resultados e Discussão

4.1. Throughput da Aplicação

A Figura 1 apresenta o throughput médio obtido pelos protocolos TCP e R-UDP nos três cenários avaliados.

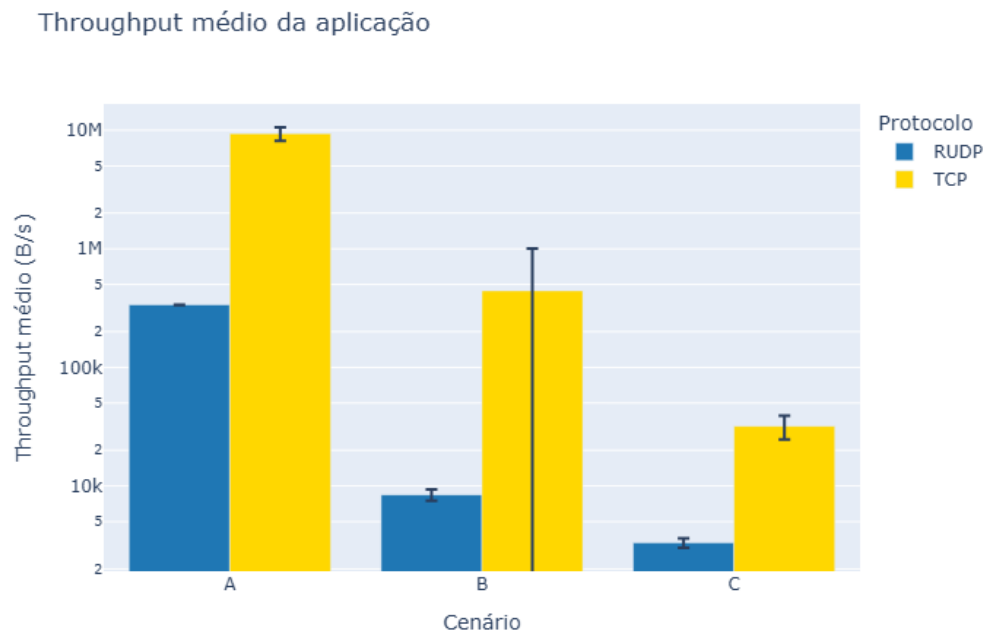


Figura 1. Throughput médio da aplicação para os protocolos TCP e R-UDP.

Observa-se que o TCP apresentou throughput superior ao R-UDP em todos os cenários analisados. No Cenário A, caracterizado por baixa latência e ausência de perdas, o TCP atingiu throughput médio de aproximadamente 9,35 MB/s, enquanto o R-UDP alcançou cerca de 337 KB/s.

À medida que as condições da rede se tornaram mais severas, ambos os protocolos sofreram degradação de desempenho. Entretanto, a redução foi significativamente mais acentuada para o R-UDP. No Cenário C, com atraso de 100 ms e perda de 20

Esse comportamento está diretamente relacionado ao aumento das retransmissões exigidas pelo mecanismo Go-Back-N implementado na camada de aplicação.

4.2. Tempo de Transferência

A Figura 2 apresenta o tempo médio necessário para concluir a transferência do arquivo em cada cenário.

Os resultados demonstram que o tempo de transferência aumentou progressivamente com a degradação das condições da rede. No Cenário A, ambos os protocolos concluíram a transferência rapidamente, embora o TCP tenha apresentado menor tempo médio.

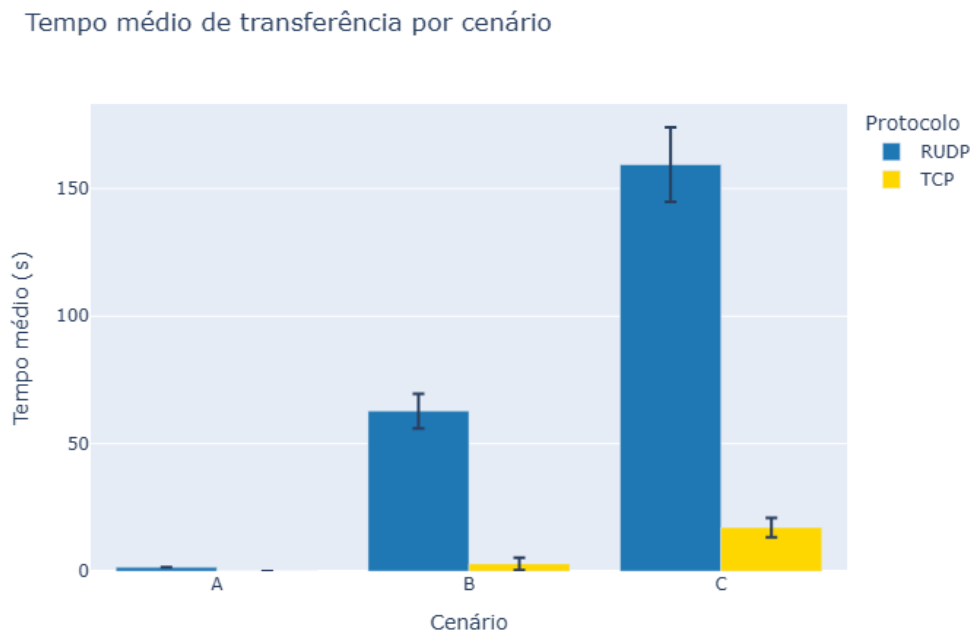


Figura 2. Tempo médio de transferência para os protocolos TCP e R-UDP.

Nos Cenários B e C, o impacto das perdas tornou-se mais evidente. O protocolo R-UDP apresentou tempos médios de aproximadamente 62,8 segundos e 159,4 segundos, respectivamente. Em contraste, o TCP concluiu as transferências em aproximadamente 2,88 segundos e 17,12 segundos.

Esses resultados evidenciam o custo das retransmissões sucessivas exigidas pelo mecanismo Go-Back-N quando ocorrem perdas de pacotes e atrasos elevados.

4.3. Retransmissões do Protocolo R-UDP

A Figura 3 apresenta a quantidade média de retransmissões realizadas pelo protocolo R-UDP durante os experimentos.

Observa-se que não ocorreram retransmissões no Cenário A, uma vez que não havia perda de pacotes na rede. Entretanto, à medida que a taxa de perda aumentou, o número de retransmissões cresceu significativamente.

No Cenário B foram observadas aproximadamente 208 retransmissões em média, enquanto no Cenário C esse valor ultrapassou 500 retransmissões. Esse comportamento é esperado devido ao funcionamento do algoritmo Go-Back-N, que exige a retransmissão de todos os pacotes da janela a partir do primeiro pacote não confirmado.

O aumento das retransmissões explica diretamente a redução do throughput e o aumento do tempo de transferência observados nas seções anteriores.

4.4. Overhead de Rede

A Figura 4 apresenta o overhead médio observado na rede durante as transmissões.

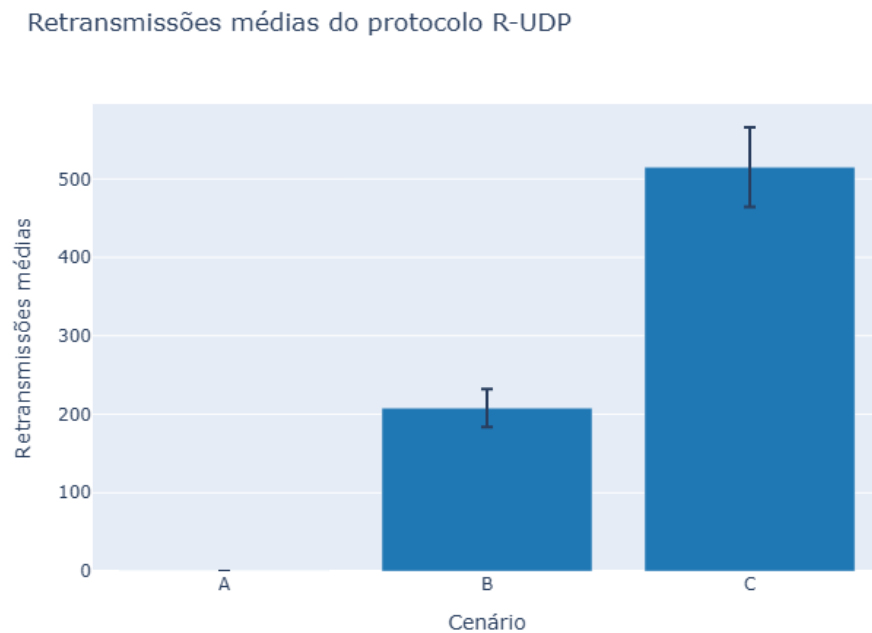


Figura 3. Quantidade média de retransmissões realizadas pelo protocolo R-UDP.

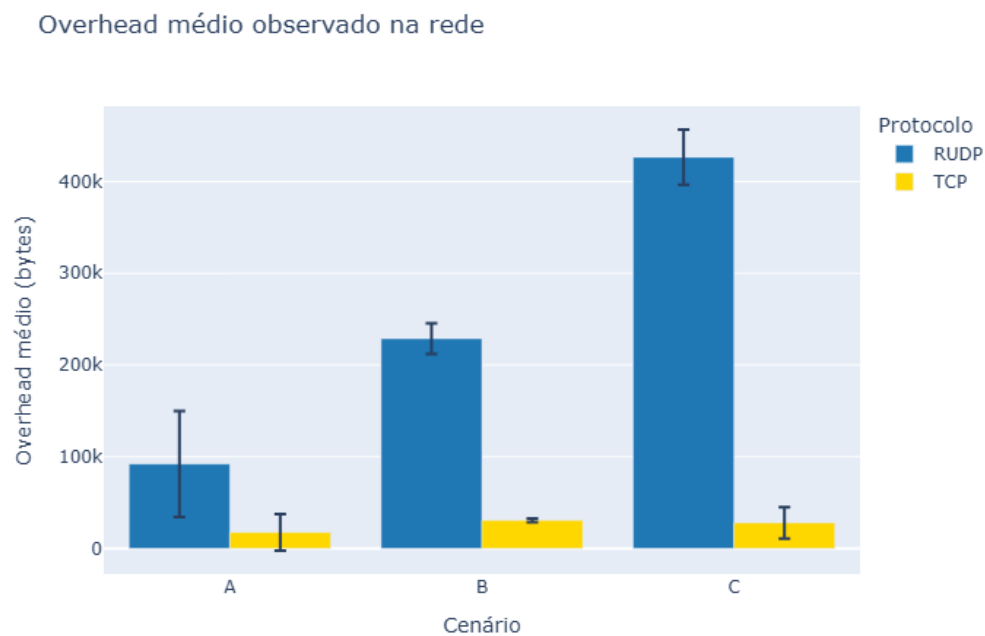


Figura 4. Overhead médio observado na rede.

O overhead foi calculado como a diferença entre a quantidade total de bytes observados na rede e o tamanho original do arquivo transmitido.

Os resultados mostram que o TCP manteve overhead relativamente estável em

todos os cenários. Em contrapartida, o R-UDP apresentou crescimento expressivo do overhead conforme as condições da rede se tornaram mais adversas.

Mesmo no Cenário A, onde não ocorreram perdas, o protocolo R-UDP gerou overhead adicional devido aos cabeçalhos dos pacotes, mensagens de confirmação e metadados da aplicação. Nos Cenários B e C, as retransmissões provocaram aumento significativo da quantidade total de dados trafegados.

4.5. Quantidade de Pacotes Observados

A Figura 5 apresenta a quantidade média de pacotes observados durante os experimentos.

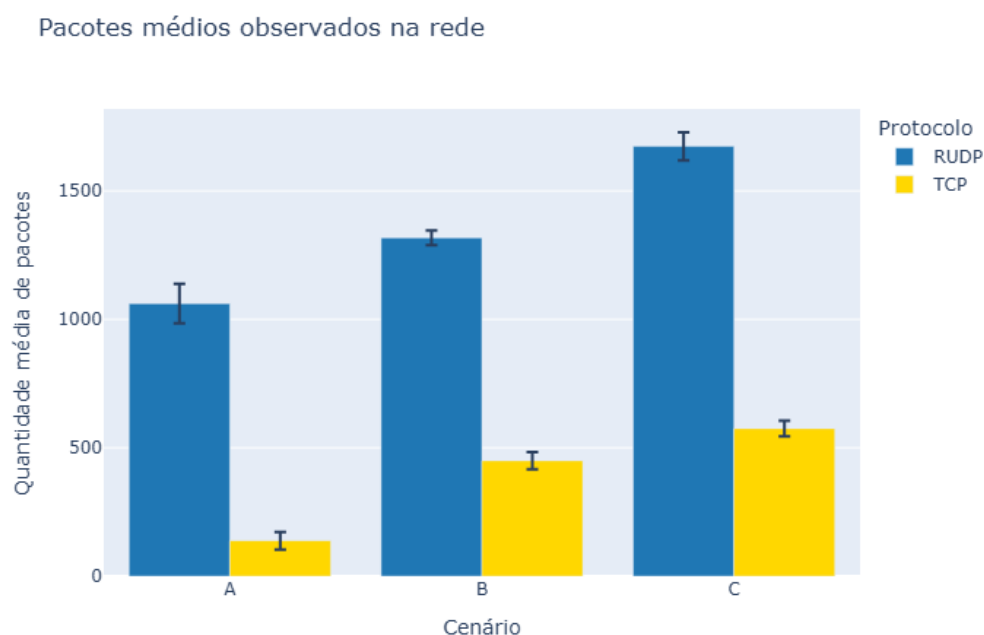


Figura 5. Quantidade média de pacotes observados na rede.

Observa-se que o protocolo R-UDP gerou um número substancialmente maior de pacotes em todos os cenários analisados.

No Cenário A foram observados aproximadamente mil pacotes para o R-UDP, enquanto o TCP utilizou menos de duzentos pacotes para transmitir o mesmo arquivo. Nos Cenários B e C essa diferença tornou-se ainda mais evidente, refletindo o impacto das retransmissões necessárias para garantir a entrega confiável dos dados.

Os resultados demonstram que a implementação do mecanismo de confiabilidade na camada de aplicação aumenta significativamente o volume de tráfego gerado na rede.

4.6. Throughput Observado na Rede

A Figura 6 apresenta o throughput calculado a partir das capturas realizadas com o tcp-dump.

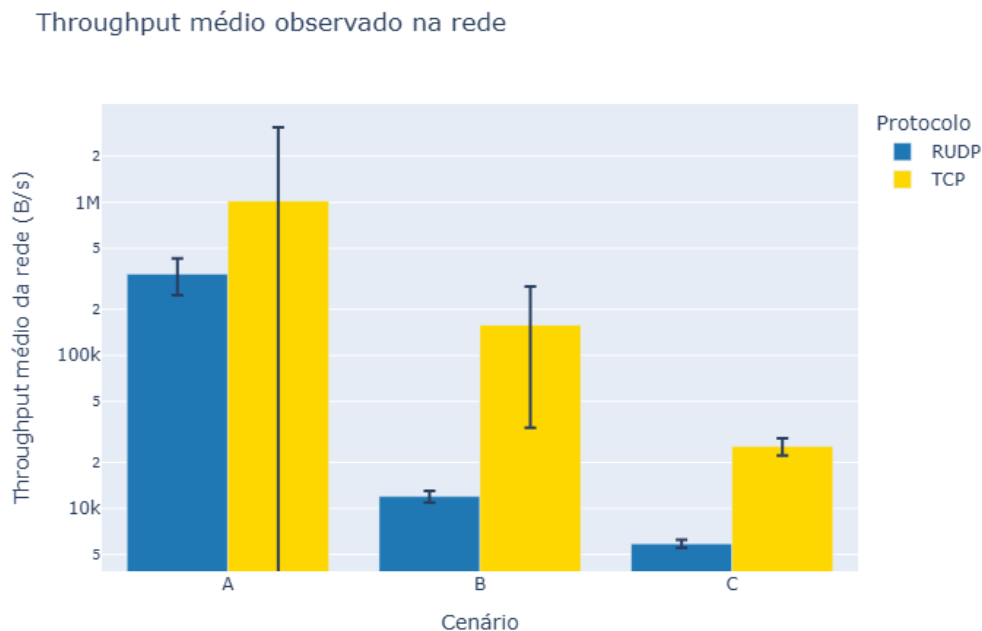


Figura 6. Throughput médio observado na rede.

Os resultados obtidos a partir das capturas de rede seguem a mesma tendência observada no throughput da aplicação. Em todos os cenários o TCP apresentou desempenho superior ao protocolo R-UDP.

A redução progressiva do throughput com o aumento da latência e da perda de pacotes evidencia o impacto das condições adversas da rede sobre ambos os protocolos. Entretanto, a degradação foi mais severa para o R-UDP devido ao crescimento das retransmissões e do overhead associado ao mecanismo Go-Back-N.

4.7. Comparação entre Throughput da Aplicação e da Rede

A Figura 7 apresenta uma comparação entre o throughput medido pela aplicação e o throughput observado diretamente na rede.

Observa-se que o throughput medido na rede difere do throughput observado pela aplicação, uma vez que as capturas incluem não apenas os dados úteis transmitidos, mas também cabeçalhos de protocolos, confirmações de recebimento, retransmissões e demais mensagens de controle.

Essa diferença é particularmente evidente no protocolo R-UDP, especialmente nos cenários com perda de pacotes, nos quais o volume de retransmissões aumenta significativamente. Como consequência, uma parcela considerável do tráfego observado na rede não corresponde diretamente aos dados úteis entregues à aplicação.

Os resultados reforçam a importância de analisar simultaneamente métricas de aplicação e métricas de rede para compreender o comportamento completo dos protocolos avaliados.

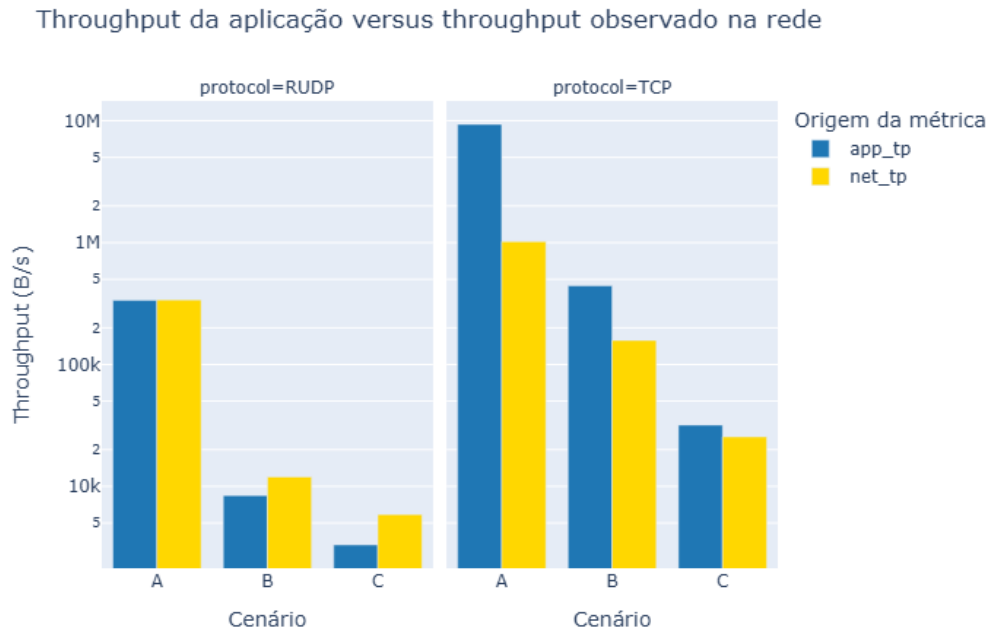


Figura 7. Comparação entre throughput da aplicação e throughput observado na rede.

5. Síntese dos Resultados

A Tabela 3 apresenta os valores médios obtidos para throughput, tempo de transferência e retransmissões em cada cenário avaliado.

Tabela 3. Resumo dos resultados experimentais

Cenário	Protocolo	Throughput Médio (B/s)	Tempo Médio (s)	Retransmissões
A	TCP	9.346.582	0,06	—
A	R-UDP	337.098	1,56	0
B	TCP	442.416	2,88	—
B	R-UDP	8.427	62,80	208
C	TCP	31.874	17,12	—
C	R-UDP	3.311	159,40	515

Os resultados demonstram que o TCP apresentou desempenho superior em todos os cenários avaliados. Embora o protocolo R-UDP tenha garantido a entrega confiável dos dados, o custo associado às retransmissões tornou-se significativo à medida que a latência e a taxa de perda aumentaram.

6. Conclusão

Este trabalho apresentou a implementação e avaliação experimental de um protocolo Reliable UDP (R-UDP) baseado no mecanismo de janela deslizante Go-Back-N. O protocolo foi desenvolvido em Python e comparado ao TCP em diferentes condições de rede simuladas por meio da ferramenta NetEm. Os experimentos demonstraram que ambos os

protocolos foram capazes de realizar a transferência correta dos arquivos nos cenários avaliados. Entretanto, observou-se que o TCP apresentou desempenho superior em todas as métricas analisadas, incluindo throughput, tempo de transferência e eficiência da utilização da rede.

Os resultados mostraram que o aumento da latência e da perda de pacotes impacta significativamente o protocolo R-UDP. Nos cenários mais severos, o mecanismo Go-Back-N gerou um elevado número de retransmissões, resultando em aumento do overhead, crescimento do volume de tráfego observado na rede e redução expressiva do throughput efetivo. A análise das capturas de rede confirmou que o R-UDP produz quantidade significativamente maior de pacotes e bytes trafegados quando comparado ao TCP, principalmente devido às retransmissões necessárias para garantir a confiabilidade da comunicação.

Apesar das limitações observadas, o desenvolvimento do protocolo permitiu compreender na prática diversos conceitos fundamentais de Redes de Computadores, incluindo controle de erros, confirmações de recebimento, retransmissão por timeout, checksum e mecanismos de janela deslizante. Como trabalhos futuros, pretende-se evoluir a implementação do protocolo, incorporando melhorias no tratamento de perdas, mecanismos mais eficientes de recuperação de erros e estratégias de retransmissão seletiva, buscando reduzir o overhead observado e melhorar o desempenho em condições adversas de rede.

Referências

TANENBAUM, A. S.; WETHERALL, D. *Computer Networks*. 5th ed. Pearson, 2011.